项目学习指南 · 学生视角

视觉跟随云台

看人 → 转头 —— 我们实际做出来的那个系统

这份文档是什么	
对象	我们实际完成的「视觉跟随云台」项目（Jetson + STM32 + LX-16A）
内容	把项目里真实用到的每个知识点拆开：要学什么、学到什么程度、怎么验证
特点	每个知识点都锚定到项目里的具体文件、具体命令、具体调试过程
结构	项目回顾 → 知识全景 → 七层拆解 → 难点复盘 → 自评 → 进阶
配套	代码：github.com/ZELDA-LINK-F/edge-ai-gimbal

不是泛泛的 AI 学习路线，而是这个项目本身的知识拆解。

你做过的东西，值得被系统地讲清楚一遍。

目 录

01 第一章 项目回顾 —— 我们到底做了什么（架构 / 数据流 / 成果）

02 第二章 知识全景 —— 七层结构，每层对应哪个文件

03 第三章 逐层拆解 —— 七层各自的知识点、深度要求、验证方法

04 第四章 难点复盘 —— 8 个真实难点，每个提炼一条方法论

05 第五章 能力自评 —— 对着项目自测，知道自己差在哪

06 第六章 进阶思考 —— 如果重做一遍，会怎么做

07 第七章 文件导读 —— 每个源文件在讲什么

08 附录 —— 命令速查 / 调试清单 / 术语

第一章　项目回顾

1.1 这个项目做了什么

一句话：摄像头看到人，云台自动转过去，让目标始终保持在画面中央。

完整链路是这样跑的：

```
摄像头拍到一帧 (1280x720)
      ↓  180 度翻转修正
YOLOv8n 识别出人，给出框 (x1,y1,x2,y2)
      ↓
算出人的水平中心 cx，归一化成偏移量 offset ∈ [-1, +1]
      ↓
映射成舵机目标位置 target = 500 + offset × 100
      ↓
平滑滤波 pan = pan + (target - pan) × 0.25
      ↓
串口发文本命令 "S1,496,200\r\n"
      ↓
STM32 解析命令，编码成 LX-16A 二进制帧 (10 字节)
      ↓
舵机转动 → 云台转向目标
```

1.2 三层硬件分工

层	硬件	职责	为什么是它
感知+决策	Jetson Orin Nano Super	摄像头采集、YOLO 推理、偏移计算	需要 GPU 跑神经网络
协议转换	STM32F103VET6	收文本命令、编二进制帧、驱动总线	实时性、微秒级响应
执行	LX-16A 总线舵机 x2	实际转动	总线级联、带位置反馈

1.3 实测成果

指标	数值	怎么测出来的
检测帧率	37 FPS	follow.py 日志里的 27ms/帧
检测准确率	100%	连续 3509 帧全部检出人
舵机定位精度	±1 / 1000	指令位置 vs read_pos 读回值

端到端延迟	< 50ms	从采集到机械动作
舵机跟随幅度	109 单位（约 26 度）	人横向走动时舵机摆动的范围
固件体积	4.5 KB	arm-none-eabi-size 输出

1.4 项目时间线

阶段	做了什么	卡在哪
M1	写固件、编译、烧进板子	FlyMcu 手动烧录太慢
M2	串口命令通信（PING → PONG）	第一版固件丢字符
M3	LX-16A 舵机驱动	协议帧多传 2 字节，舵机完全不动
M4	双舵机 ID 管理	两个舵机默认都是 ID 1
M5	Jetson 控制舵机（全链路）	串口设备识别
M6	云台跟随（YOLO → 舵机）	摄像头对着天花板 / 装反了

注意：M3 是整个项目耗时最长的一步。协议错了 2 个字节，舵机一声不吭 —— 详见第四章。

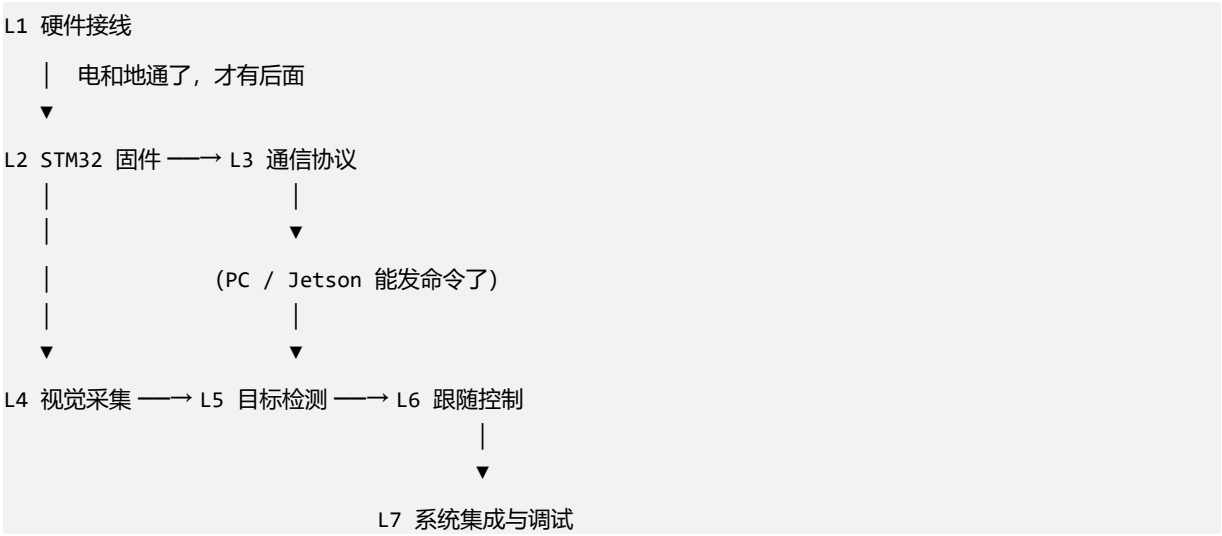
第二章　知识全景

整个项目可以拆成七层。每层解决一个独立的问题，层与层之间通过明确的接口连接。

2.1 七层结构

层	名称	解决的问题	对应文件
L1	硬件与接线	电怎么供、地怎么共、信号怎么接	（接线，无文件）
L2	STM32 固件	怎么让单片机跑起来、驱动外设	main.c / startup.c / linker.ld
L3	通信协议	两台设备怎么可靠地对话	main.c 解析器 + servo_send_frame
L4	视觉采集	怎么把画面变成数字图像	camera.py
L5	目标检测	怎么在图像里找到人	detect.py
L6	跟随控制	怎么把检测结果变成动作	follow.py / servo.py
L7	系统集成	怎么把七层拼起来并调试	全部 + flash.ps1

2.2 依赖关系



调试顺序也要遵循这个依赖：

先确认 L1 接对（LED 亮不亮）→ 再确认 L2 跑起来（串口打印）→ 再确认 L3 通（PING 有回应）→ 再确认 L4 有画面 → L5 能检测 → L6 能跟 → 最后 L7 整体联调。

本项目最大的教训之一：一定要按这个顺序查，不要跳步。

2.3 每层要学到什么程度

层	核心知识点数量	建议深度	说明
L1 硬件	约 6 个	★★ 会用	能自己接线、能解释共地
L2 STM32	约 9 个	★★★ 精通	这是项目的硬核部分，建议深入
L3 协议	约 7 个	★★★ 精通	协议能力通用性最强
L4 视觉	约 7 个	★★ 会用	会用 OpenCV 和 GStreamer 即可
L5 检测	约 7 个	★★★ 精通	YOLO 部署是 AI 落地核心
L6 控制	约 7 个	★★ 会用	理解映射、滤波、限位
L7 集成	约 7 个	★★★ 精通	这是工程师最值钱的能力

建议：L2 / L3 / L5 / L7 四层往精通走，其余六成会用就够了。

第三章　逐层拆解

本章按七层逐一拆解。每一层回答四个问题：

- 这一层要解决什么问题
- 需要掌握哪些知识点，各自到什么深度
- 在我们项目里具体是怎么做的
- 怎么验证自己真的学会了

深度标注：★ 了解　★★ 会用　★★★ 精通

L1 硬件与接线

目标：让电供得上、地共得通、信号接得对。这一层不对，上面所有层都是白搭。

知识点与深度

知识点	深度	在项目里怎么用
供电分离	★★★	舵机用 7.4V 电池独立供电，Jetson 和 STM32 用 USB 5V
共地	★★★	面包板负极轨把电池负极、舵机 GND、板子 GND 接到一起
面包板结构	★★★	知道一排 5 个孔连通、不同排不连通
信号线连接	★★★	舵机 SIG 接 PA2（发送）+ PA3（接收/上拉）
引脚识别	★★★	会看板子丝印，JP4 第 5 行靠边是 GND，JP5 第 3/4 行靠中间是 PA2/PA3
排线保护	★★	FPC 硬排线怕弯折，走线要靠近转轴

我们项目里具体怎么做

- 电池正极 → 舵机 VIN；电池负极 → 舵机 GND + 板子 GND（JP4 第 5 行靠边）
- 舵机白线（SIG）→ 面包板同一排 → 两根杜邦线分别接 PA2、PA3
- 为什么舵机要独立供电：堵转电流 2~3A，USB 扛不住，会拉垮主板

怎么验证自己会了（做完才算是会用）

1. 通电后舵机 LED 常亮（不是闪、不是灭）

2. 用手转舵机输出盘，有明显阻力（说明电机被装载）

3. 量一下电池到板子 GND 是否导通（万用表蜂鸣档）

4. 解释：为什么两套电源必须共地？不共地会怎样？

踩过的坑

- PA2 / PA3 插在面包板不同排 —— 这是本项目卡了很久的坑
- 只接 TX 不接 RX，总线没有上拉，信号拉不高
- 舵机从板子 5V 取电，导致板子重启

L2 STM32 固件

目标：不借助任何库，让 STM32 跑起来、让两个串口工作、让时间可计量。

知识点与深度

知识点	深度	在项目里怎么用
启动流程	★★★	上电从 0x08000000 读栈指针，从 0x08000004 读入口地址
中断向量表	★★	startup.c 里的函数指针数组，SysTick 用到了它
链接脚本	★★★	linker.ld 定义 512K Flash 和 64K RAM 的布局
寄存器映射	★★★	用结构体把 0x40021000 这类地址变成 RCC->CR 这样的写法
volatile	★★★	寄存器变量必须加，否则编译器会优化掉读写
时钟树	★★★	HSE 8MHz × 9 = 72MHz，APB1 分频后 36MHz
GPIO 八种模式	★★★	推挽/开漏/上拉/浮空，本项目动态切换 PA2 模式
USART 配置	★★★	USART1 在 72MHz 总线上 BRR=625，USART2 在 36MHz 总线上 BRR=312
SysTick	★★	9MHz 下 LOAD=9000-1 得到 1ms

我们项目里具体怎么做

- clock_init()：开 HSE、配 PLL、切 SYSCLK、设 FLASH_ACR 等待周期
- gpio_usart_init()：PA9/PA10 给 USART1，PA2/PA3 给 USART2
- pa2_drive(1/0)：发送时切 AF 推挽，发完切浮空输入（高阻）
- systick_init()：CTRL=3（使能+中断+HCLK/8），LOAD=9000-1

怎么验证自己会了（做完才算是会用）

1. 说出 STM32 上电到进 main() 之间发生了什么
2. 手算：72MHz 下 115200 的 BRR 是多少？36MHz 下呢?
3. 解释：为什么 PA2 要在发送后切成高阻态?
4. 画出 PA2 的 GPIO 配置寄存器（CRL 的 4 个 bit）对应的四种模式
5. 如果你要加一个 PA5 输出方波，改哪几行?

踩过的坑

- 时钟没配对 → 串口全是乱码
- 没开外设时钟 → 配置了寄存器但外设不工作
- GPIO 模式选错 → 推挽和开漏搞混会总线冲突
- 主循环里加 delay_ms(1) → 串口严重丢字符

L3 通信协议

目标：让 Jetson 和 STM32、STM32 和舵机，都能可靠地对话。这是本项目的灵魂。

知识点与深度

知识点	深度	在项目里怎么用
文本协议设计	★★★	S1,500,1000 这种命令，可读性强好调试
二进制协议设计	★★★	LX-16A 的 55 55 ID Length Cmd Params Checksum
校验和	★★★	Checksum = ~(ID+Length+Cmd+参数) & 0xFF
Length 字段的语义	★★★	Length = ID + Cmd + 参数 + 校验和（本项目的最大坑）
帧头识别	★★	连续两个 0x55 表示包开始

多设备寻址	★★	每个舵机有 ID，254 是广播
应答机制	★★	读指令需要设备返回数据，要预留接收窗口
静默失败	★★★	协议错了设备不报错，直接丢弃 —— 这是最难查的

我们项目里具体怎么做

- Jetson → STM32：PING / S<id>,<pos> / R<id> / ID<old>,<new> 等文本命令
- STM32 → 舵机：servo_send_frame() 编码 10 字节二进制帧
- servo_move()：位置 4 字节参数（角度 2+时间 2），Length=7
- 行缓冲 + 非阻塞轮询：保证高波特率不丢字符

怎么验证自己会了（做完才算是会用）

1. 手算：55 55 01 07 01 F4 01 E8 03 的校验和是多少？（答案 0x16）
2. 解释：为什么 Length 多算 2 字节，舵机就完全不动?
3. 用官方例子验证你的编码函数输出是否一致
4. 设计一个读温度的文本命令，并说明 Jetson 端怎么解析应答
5. 解释：为什么文本协议适合调试，二进制协议适合设备间通信?

踩过的坑

- 凭记忆写协议 → 多传 2 字节，舵机静默丢弃（耗时最长的一个坑）
- 校验和算法写错 → 同样静默失败
- 没考虑大小端 → 多字节参数顺序反了
- 发送后立刻打印日志 → 舵机应答的字节被丢掉了

L4 视觉采集

目标：把现实世界变成一帧帧可以处理的数字图像。

知识点与深度

知识点	深度	在项目里怎么用
图像表示	★★★	图像是 H x W x 3 的数组，OpenCV

		里是 BGR 顺序
分辨率与帧率	★★★	1280x720@30fps 表示什么，和延迟什么关系
GStreamer 管线	★★	用 ! 把 element 串起来，appsink 交给 Python
CSI 摄像头	★★	nvarguscamerasrc 走硬件 ISP，比 USB 延迟低
自动曝光收敛	★★★	前几十帧不可用，要预热后取最后一帧
OpenCV 基础	★★★	VideoCapture / read / rotate / resize / cvtColor
画面方向	★★	摄像头装反了要旋转 180 度

我们项目里具体怎么做

- camera.py 提供 photo / video / frames 三种取帧方式
- PREHEAT_FRAMES = 90：拍 90 帧预热，只保留最后一帧
- follow.py 里用 cv2.rotate(frame, cv2.ROTATE_180) 做翻转修正
- drop=1 max-buffers=2：丢弃积压帧，保证低延迟

怎么验证自己会了（做完才算是会用）

1. 用 OpenCV 读一张图，打印 shape
2. 写一条 GStreamer 管线，把画面存成图片
3. 解释：为什么拍出来的第一张图是全黑的?
4. 解释：drop=1 和 max-buffers=2 分别解决什么问题?

踩过的坑

- 不预热 → 全黑或纯噪声图
- 摄像头装反 → 人和舵机转向相反（本项目实测过）
- 分辨率开太大 → 推理变慢、内存吃紧
- 忘了解码格式转换 → 拿到的是 NV12 不是 BGR

L5 目标检测

目标：在图像里找到人，并给出它的位置。这是 AI 真正落地的那一步。

知识点与深度

知识点	深度	在项目里怎么用
检测任务定义	★★★	分类（是什么）+ 定位（在哪里）
YOLO 基本原理	★★	网格划分、置信度、NMS 去重
模型规模选择	★★★	n/s/m/l/x，越大越准越慢，本项目用 n
COCO 类别	★★	80 类，person 是第 0 类
推理流程	★★★	预处理 → 推理 → 后处理，每一段都可能成为瓶颈
置信度阈值	★★★	CONF=0.4，阈值越高漏检越多
模型部署	★★★	PyTorch → ONNX → TensorRT，FP16 加速 1.39 倍
引擎元数据丢失	★★	trtexec 导出的引擎没有类别名，要手动映射

我们项目里具体怎么做

- detect.py 的 load() 和 detect_frame()，返回结构化检测结果
- follow.py 里筛选 person 类，取面积最大的那个
- 640 输入时 27.3ms → 19.6ms；720 输入时没变化（瓶颈在预处理）

怎么验证自己会了（做完才算是会用）

1. 用 YOLOv8n 检测一张有人的图，打印所有结果
2. 把 conf 从 0.1 调到 0.9，观察检测数量怎么变
3. 解释：为什么跟随只取面积最大的 person?
4. 解释：为什么 TensorRT 在 640 输入时加速明显、720 时没有?
5. 导出一次 ONNX，用 trtexec 构建引擎，记录耗时

踩过的坑

- 模型太大 → 帧率不够，跟随有延迟
- 阈值太高 → 人走远一点就丢了

- 没过滤类别 → 把椅子也当成目标
- 输入尺寸和训练尺寸不一致 → 精度下降

L6 跟随控制

目标：把检测结果变成平滑、安全、可用的机械动作。

知识点与深度

知识点	深度	在项目里怎么用
坐标映射	★★★	画面 x 坐标 → 归一化偏移 → 舵机位置
指数平滑	★★★	pan = pan + (target - pan) x alpha，alpha=0.25
安全限位	★★★	PAN_MIN/PAN_MAX = 400/600，保护摄像头排线
开环控制	★★	本项目只发命令不闭环调节，舵机自己到位
角度换算	★★	位置 0~1000 对应 0~240 度，1 单位 = 0.24 度
快速发送	★★	跟随循环里不等应答，直接写串口
丢目标处理	★★	没人时慢慢回中位，而不是急停

我们项目里具体怎么做

- follow.py 的 step()：检测 → 算偏移 → 映射 → 平滑 → 限位 → 发送
- servo.py 的 clamp()：把位置限制在 400~600，超限时打印提示
- 每 2 秒清一次接收缓冲，避免固件回显堆积

怎么验证自己会了（做完才算是会用）

1. 推导：如果 alpha=0.25，目标阶跃 100 单位，几步能到 90 以上?
2. 把 SMOOTH 从 0.1 调到 1.0，观察跟随效果差异
3. 设计一个限位函数，超限返回边界值并提示
4. 解释：为什么没人时要慢慢回中，而不是直接急停?

踩过的坑

- alpha 太大 → 舵机抖动、异响
- alpha 太小 → 跟随滞后明显
- 没限位 → 舵机顶到机械限位，可能扯断排线
- 等应答再发下一帧 → 跟随循环变慢

L7 系统集成与调试

目标：把七层拼成一个能跑的系统，并且出问题时能快速定位。这是工程师最值钱的能力。

知识点与深度

知识点	深度	在项目里怎么用
分层调试	★★★	从电源 → 通信 → 协议 → 视觉 → 算法，逐层验证
诊断工具设计	★★★	在固件里加 LOOP / SCAN / BAUD 三个自测命令
日志与回显	★★★	关键节点打印 TX 十六进制、OK、位置读数
故障树	★★★	把「不工作」拆成可验证的子问题
边界条件	★★★	位置 0/1000、断线、电池耗尽、断电恢复
串口设备识别	★★	Jetson 上 /dev/ttyCH341USB1 是 STM32
自动化脚本	★★★	flash.ps1 用 Win32 API 自动点 FlyMcu
版本管理	★★	Git 提交、GitHub 托管

我们项目里具体怎么做

- LOOP：PA2 发 0x55，PA3 收 —— 5 秒定位线路断点
- SCAN：遍历 ID 1~20 找在线舵机
- BAUD：遍历 9600~1000000 找舵机波特率
- flash.ps1：改 INI + 窗口置顶 + 真实鼠标点击 + 检查日志

怎么验证自己会了（做完才算是会用）

1. 画一张「舵机不动」的故障树，至少 5 个分支
2. 给固件加一个新诊断命令（比如读电压）
3. 故意插错一根线，然后按流程定位它
4. 用 Git 提交一次修改，写清楚 commit message
5. 写一个脚本，一条命令完成「编译 + 烧录 + 验证」

踩过的坑

- 一上来就改代码，不先定位问题
- 没有诊断手段，只能靠猜
- 不写文档，一周后自己都忘了怎么搭的
- 电池耗尽误判成通信故障（本项目踩过）

第四章　难点复盘

本项目遇到 8 个真实难点。比「怎么解决的」更重要的是「提炼出了什么方法」。

1. 舵机完全不动（耗时最长）

现象：LED 常亮、STM32 回 OK，但舵机不转不应答；试了 20 多种波特率、时序、接线，全部无效

原因：凭记忆写协议，位置指令多加了速度 2 字节；Length 字段与参数长度不匹配，舵机静默丢弃整个包

解决：逐字节对照官方文档；修正为 4 字节参数、Length=7；帧与官方例子完全一致后舵机立即恢复

方法：协议实现前必须先读文档、先用官方例子做单元测试

2. 摄像头从系统里消失

现象：系统更新后 /dev/video0 不见了；i2cdetect 什么都看不到

原因：apt upgrade 重置了 extlinux.conf；DEFAULT 从 JetsonIO 变回 primary，摄像头 overlay 没加载

解决：改回 DEFAULT JetsonIO 后重启；建立习惯：系统更新后先查这个文件

方法：出问题先查配置有没有被覆盖，再怀疑硬件

3. 舵机总线收不到信号

现象：舵机有电，但完全不响应；STM32 发送正常

原因：PA2 和 PA3 插在面包板不同横排；两排不连通，信号根本没接上

解决：插到同一横排；LOOP 回环测试立即通过

方法：用「发什么收什么」的回环自测，比万用表更快定位断点

4. FlyMcu 无法自动化

现象：UI Automation 找不到控件；SendKeys 无反应，BM_CLICK 无反应；真实鼠标点击也无反应

原因：Delphi 程序不暴露控件；窗口被其他窗口遮挡，点击落在别处

解决：Win32 枚举子窗口找到按钮坐标；窗口置顶 + 真实鼠标事件；用日志文件判断结果

方法：GUI 自动化失败时先截图看屏幕实际状态

5. FPC 排线易损

现象：摄像头用硬质排线，云台转动会反复弯折；断裂后无法修复

原因：FPC 耐扭转但不耐弯折；全角度云台不可行

解决：软件限位 400~600（约 ±24 度）；走线靠近转轴，让转动变成扭转；固定排线根部，预留服务环

方法：软件限位也是硬件保护的一部分

6. 舵机突然全部不响应

现象：连续测试 2 分钟后两个舵机都没反应

原因：7.4V 电池耗尽；电池没电时舵机静默不响应，不报错

解决：充电后恢复；建立电量检查习惯

方法：电池耗尽的症状和通信故障极像，要先排除

7. 读位置出现 65534

现象：舵机转到 0 时读回 65534

原因：顶到限位有 -2 的负过冲；无符号解析变成 0xFFFE

解决：按有符号 16 位解析：v-65536 if v>32000；显示时处理负数

方法：边界条件必须单独测试

8. PowerShell 远程命令引号丢失

现象：ssh 执行 python -c 时报语法错误

原因：PowerShell 处理引号和变量，破坏了命令

解决：把脚本写成文件，scp 上传后执行

方法：复杂的远程命令不要内联，写成文件

这 8 条方法论，比任何技术细节都更通用：

方法论	适用场景
协议先读文档、先做对照测试	任何二进制协议 / 文件格式 / 通信协议
先查配置再怀疑硬件	嵌入式设备异常
用回环自测定位线路	任何有线通信
GUI 自动化先截图	任何桌面自动化
软件限位保护硬件	任何有运动范围的机械结构
先排除电源再查通信	任何设备不响应
边界条件单独测	任何数值处理
远程命令写成文件	跨平台开发

第五章　能力自评

对着本项目自测。每项 1~5 分：1=不会，3=能看懂，5=能独立重写。

5.1 分项自评

层	自评项	分数
L1	能独立完成电池、舵机、板子的接线并解释共地	
L1	能识别 JP4/JP5 上的 GND、PA2、PA3	
L2	能说出 STM32 上电到进 main 的完整过程	
L2	能手写 RCC/GPIO/USART 的寄存器配置	
L2	能解释 volatile 和 PA2 动态切模式的原因	
L3	能设计一个带校验和的帧格式	
L3	能手算 LX-16A 帧的校验和	
L3	能解释 Length 字段的准确含义	
L4	能写 GStreamer 管线并解释每个 element	
L4	能解释为什么要预热 90 帧	
L5	能部署 YOLO 到 Jetson 并解释推理流程	
L5	能导出 ONNX 并用 trtexec 构建引擎	
L5	能分析推理瓶颈在预处理还是模型	
L6	能实现坐标映射 + 指数平滑 + 限位	
L6	能调参并解释每个参数的影响	
L7	能设计诊断命令并用它定位问题	
L7	能画故障树并逐项排除	

L7	能把整个项目用一条命令构建和烧录	

5.2 总分对照

总分	水平	说明
18 ~ 40	刚做完，还没消化	建议按本指南重做一遍关键部分
41 ~ 60	基本掌握	能改参数、能做小改动
61 ~ 78	熟练掌握	能独立完成同类项目
79 ~ 90	优秀	能优化、能设计新方案

5.3 自查三问

问题	含义
能不看资料讲清楚吗?	检验是真的懂了，还是只是抄过一遍
能改需求吗?	检验掌握程度：比如把水平跟随改成上下跟随
能排查故障吗?	检验调试能力：给你一个不动的舵机，你的排查顺序是什么

三问都答得上，才算真的「学会了」这个项目。

第六章　进阶思考

做完不等于做好。下面这些问题，可以作为下一步的练习方向。

6.1 技术上的改进空间

方向	现状	可以怎么做
机械结构	摄像头固定，舵机空转	把摄像头装到舵盘上，做真正的云台
俯仰控制	只做了水平（ID1）	加 ID2 俯仰，做二维云台
闭环控制	开环发命令，不读位置	读位置反馈，做 PID 闭环
多目标	只跟面积最大的人	多目标跟踪（DeepSORT），锁定特定目标
响应速度	27ms 推理	INT8 量化、更小模型、异步流水线
安全限位	固定 ±24 度	根据排线余量动态配置
错误处理	简单重连	完善的断线重连、异常恢复
测试	手动测试	写自动化回归测试

6.2 如果重做一遍，你会怎么做

这是面试常问的问题，也是真正检验有没有消化的方式。

阶段	当时的做法	现在会怎么做
协议	凭记忆写，错了才查文档	先读文档、先写对照测试，再动手
烧录	手动点 FlyMcu 十几次	第一天就把自动化脚本写好
调试	遇到问题凭感觉试	先建诊断工具、先画故障树
文档	全部做完才写	开发过程中同步记录，边做边写
测试	改完手动跑一遍	关键路径写自动化测试
版本	最后才提交 Git	每完成一个功能就提交一次

6.3 可以继续深挖的方向

方向	具体内容	对应能力
RTOS	FreeRTOS 任务调度、消息队列	嵌入式进阶
PID 控制	位置闭环、参数整定	控制理论

多目标跟踪	DeepSORT / ByteTrack	计算机视觉
模型量化	INT8 校准、混合精度	AI 部署
ROS2	节点通信、TF 变换	机器人系统
硬件设计	自己画 PCB、电源设计	硬件工程

第七章　文件导读

想重新理解项目，最好的方式是逐个文件读一遍。下面说明每个文件在讲什么、重点看哪部分。

7.1 STM32 固件

文件	行数	重点看什么
main.c	约 600	寄存器定义 → 时钟配置 → GPIO/USART → SysTick → 命令解析 → 舵机帧编码
startup.c	约 60	中断向量表结构，SysTick_Handler 怎么挂上去的
linker.ld	约 40	MEMORY 和 SECTIONS，.text/.data/.bss 分别放哪
flash.ps1	约 150	Win32 API 调用、窗口置顶、鼠标事件模拟

7.2 Jetson 应用

文件	行数	重点看什么
camera.py	约 250	三种取帧方式、GStreamer 管线、预热帧
detect.py	约 200	模型加载、detect_frame 的输入输出结构
servo.py	约 230	串口封装、命令拼接、应答解析、安全限位
follow.py	约 190	主循环：检测 → 偏移 → 映射 → 平滑 → 发送

7.3 推荐阅读顺序

```
第一步：follow.py          ← 先看整体逻辑，知道数据怎么流动
第二步：servo.py           ← 看它怎么调固件
```

```
第三步：main.c（命令解析部分）← 看固件怎么响应
第四步：main.c（舵机帧编码）  ← 看协议怎么实现
第五步：camera.py + detect.py ← 看视觉部分
第六步：startup.c + linker.ld ← 看最底层
第七步：flash.ps1             ← 看工程化怎么做的
```

按这个顺序读，是从「用」到「实现」再到「原理」，符合认知规律。

附录 A：命令速查

命令	作用	示例
PING	心跳检测	→ PONG
S1,500	舵机 1 转到中位	S1,500
S1,500,1000	1 秒内转到中位	S1,500,1000
S254,500	广播（两个一起动）	S254,500
R1	读舵机 1 位置	→ 收 9 字节
ID1,2	把 ID1 改成 ID2	ID1,2
SCAN	扫描在线舵机	
LOOP	回环自测 PA2↔PA3	
BAUD	波特率扫描	

附录 B：故障排查清单

舵机不动时，按这个顺序查（来自本项目的实战经验）：

步骤	检查什么	怎么查
1	STM32 还活着吗	发 PING，看回不回 PONG
2	信号线通吗	发 LOOP，看能不能收到 0x55
3	舵机有电吗	看 LED（亮/闪/灭）+ 手转输出盘有阻力吗
4	舵机应答吗	发 R1，看有没有 8 字节应答
5	帧格式对吗	打印 TX 十六进制，和官方例子逐字节比对
6	电池还有电吗	充电试试（症状和通信故障很像）

附录 C：关键数字备忘

数字	含义
115200	两个串口的波特率
625 / 312	USART1 / USART2 的 BRR 值

72MHz / 36MHz	APB2 / APB1 总线频率
9000-1	SysTick 的 LOAD 值（得到 1ms）
0x08000000	Flash 起始地址
0x20000000	RAM 起始地址
55 55	LX-16A 帧头
254 (0xFE)	广播 ID
400 / 600	安全限位范围（约 ±24 度）
500	中位
0.24 度	1 个位置单位对应的角度
27ms / 37FPS	YOLOv8n 的推理耗时与帧率

— 指南完 —

把这个项目真正讲清楚一遍，你就真的学会了。